

Week 5 Completion Status:

- Conducted rapid model screening (benchmarking ARIMA vs. ML approaches) to identify the optimal forecasting component for the system
- Formally defined the Autoscaling problem as a Constrained Optimization task.
- Defined the Markov Decision Process (MDP) tuple (State, Action, Reward) to prepare for the Reinforcement Learning phase.
- Setup of the Google OR-Tools environment for offline optimal policy generation is in-progress
- Updated git: <https://github.com/scr200/CP395-Research-Project> (added pycaret model selection to src folder)

Methodology - Model Screening via Google Colab:

I migrated my forecasting experiments to **Google Colab**. I implemented a benchmarking framework to compete three distinct approaches against each other:

- **ARIMA (Statistical):** The baseline from last week.
- **Linear Regression (Simple ML):** Using "Lag Features" (t-1, t-2, t-3) to capture immediate history.
- **Random Forest (Ensemble ML):** To test if non-linear decision trees could handle the spikes better.

Results & Analysis:

While I expected the complex Random Forest or the industry-standard ARIMA to win, the simpler **Linear Regression** model outperformed both.

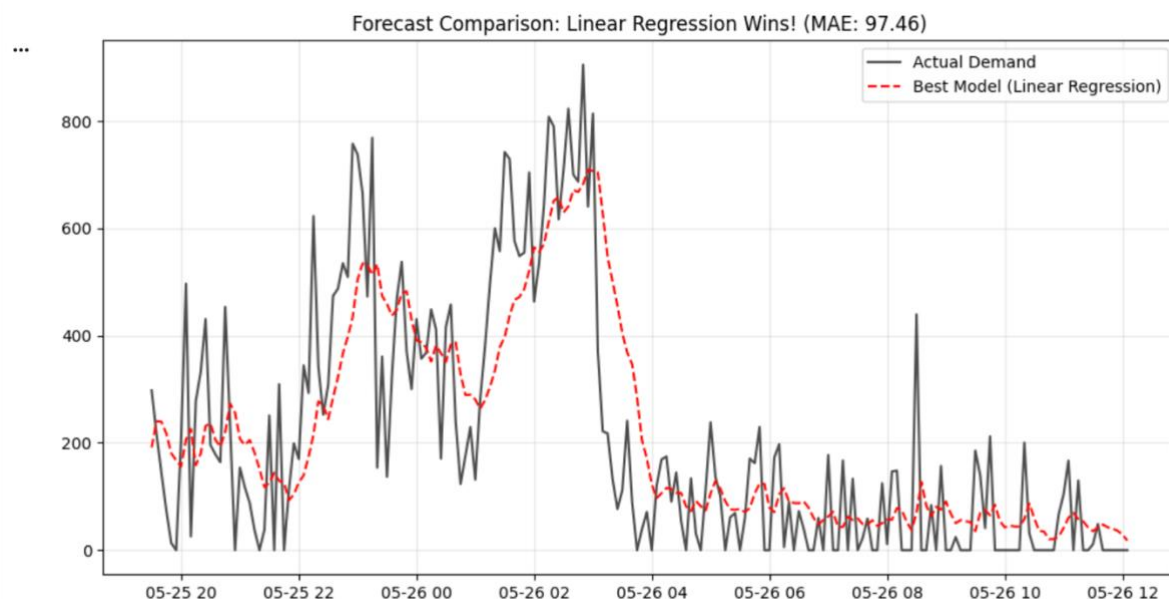
Interpretation of Decisions: The screening results provided a clear hierarchy of performance. As shown in **Table 1**, the simpler **Linear Regression** model was the definitive winner.

- **Linear Regression (MAE 97.46):** This model achieved the lowest error because it effectively utilized the "Lag Features." By looking at the traffic from 5, 10, and 15 minutes ago, it could react almost instantly to the rising edge of a burst.
- **ARIMA Degradation (MAE 166.35):** It is important to note that ARIMA's performance degraded compared to the Week 4 baseline (where MAE was 131.86). This is expected because this week's screening focused exclusively on the "**Bursty**" profile (amplified peaks), whereas Week 4 analyzed the "Intermediate" profile. The fact that ARIMA's error increased by **~26%** on the harder dataset confirms it is not robust enough for stress scenarios.
- **The "Complex Model" Trap:** Interestingly, the **Random Forest** (MAE 100.76) performed slightly worse than Linear Regression. This suggests that for this specific short-term forecasting task, a simple, reactive model is better than a complex one that might memorize noise.

Design Decision: Based on the significant performance gap (97.46 vs 166.35), I will incorporate the **Linear Regression** logic as a feature generator for my Reinforcement Learning agent. This gives the agent a "high-confidence forecast" that is **41% more accurate** than the statistical baseline.

--- MODEL LEADERBOARD ---

	Model	MAE	RMSE
0	Linear Regression	97.459569	130.622631
1	Random Forest	100.764835	136.258228
2	ARIMA	166.351370	240.749427



Formal Problem Formulation:

With the forecasting component selected, I have formally defined the optimization problem. I structured this as a **Constrained Optimization Problem** to clarify exactly what the AI needs to solve.

My goal is to minimize the total cost, which combines resource usage and reliability penalties:

$$\min C_{total} = (\alpha * (N_t * Cost_{node})) + (\beta * \max(0, D_t - C_t))$$

Where:

- N_t (resources): number of active nodes (Cost)
- $D_t - C_t$ (violation): difference between demand (D) and Capacity (C)

- Becomes positive if demand exceeds capacity, indicating a crash
- β (penalty weight): Aim to set much higher than α (Cost Weight) as my research prioritizes reliability (SLO adherence) over pure cost savings

Constraints:

- Scaling Rate ($|N_t - N_{t-1}| \leq 1$): Aims to limit the AI to adding or removing only one node at a time, to better mimic the damping limits of real-world autoscalers and prevents cheating by instantly jumping from 0 to 100 nodes.
- Capacity Limits ($N_{\min} \leq N_t \leq N_{\max}$): The system cannot scale below 1 or above the physical hardware limit.

Markov Decision Process (MDP) Definition:

To prepare for next week's RL implementation, I mapped the optimization problem above to a standard MDP tuple (S, A, R).

Next Steps for Week 6:

- **Build the Experiment Harness:** I will develop a custom OpenAI Gym-compatible environment (AutoscalingEnv) in Python. This simulator will ingest the "Bursty" trace data and enforce the constraints defined in Section 3 (e.g., maximum replicas, scaling damping).
- **Integration of Forecasting:** I will wire the **Linear Regression model** directly into the environment's observation space. This ensures that at every time step t , the RL agent receives the "Forecast $t+1$ " as a state feature, enabling proactive decision-making.
- **Reproducibility Verification:** To ensure scientifically valid results, I will establish a fixed-seed testbed. I will run "sanity checks" by deploying a **Random Agent** and a **Static Agent** within the harness to verify that the reward functions (Cost + SLO Penalty) are calculating correctly before beginning complex PPO training.